PIM IV · ADS · 4º SEMESTRE

UNIP · São José dos Campos · Novembro/2026

Nex_TI

Solução Web e Mobile de avaliação e apoio à aprendizagem para uma empresa fictícia de tecnologia educacional

Como as oito disciplinas do semestre viraram um só sistema: do problema à plataforma no ar.

Gabriel Alves Moreira
H67HJI4

Maciel Costa da Silva
R280985

Maycon Douglas Inácio Silva
H719CD3

Miguel Angel Fernandez Ortiz
H7858F9

Rafael Mesquita
H6722I0

UNIP
UNIVERSIDADE PAULISTA

github.com/MayconDIS/PIM_IV · nexti-pim-iv.vercel.app

Começo, meio e fim: a história do Nex_TI

01

Começo · o problema

- Quem é o aluno e o que o atrapalha.
- Como levantamos os requisitos (elicitação).
- Da necessidade ao teste.

02

Meio · as disciplinas

- Uma disciplina por slide: o que ela ensinou, onde usamos e a prova no sistema.
- No fim do meio, uma resposta do aluno passando pelas oito.

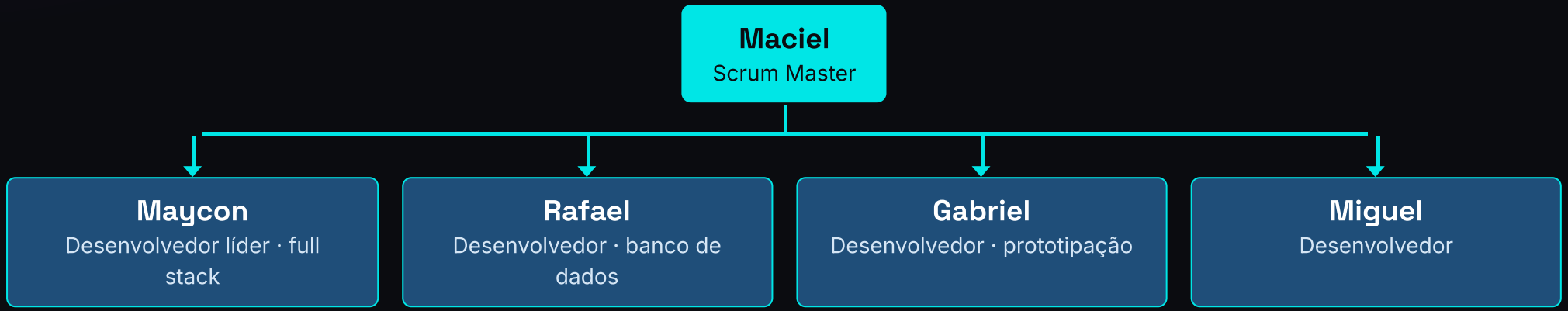
03

Fim · os resultados

- O que os testes mostraram.
- O que resolvemos e o que ainda falta medir.
- O sistema funcionando.

Empreendedorismo em TI | Relações Étnico-Raciais | Web com .NET | Mobile | Programação Aplicada em .NET | Banco de Dados | Cloud e DevOps | Projetos Ágil

O DevTeam: papéis definidos no Scrum



A · Scrum Master (Maciel): monta o quadro no Trello, organiza as sprints e o Sprint Planning e acompanha as entregas.

B · Desenvolvedor líder (Maycon): API, site, app, banco, nuvem e testes; integra os pull requests e cuida da documentação.

C · Desenvolvedores (Rafael, Gabriel e Miguel): áreas definidas no planejamento: banco de dados (Rafael) e prototipação das telas (Gabriel).

Quem começa em TI tropeça nos termos técnicos

Muitos termos novos

Logo no primeiro semestre aparecem dezenas de conceitos de lógica, banco, redes e engenharia.

Estudo pouco sistemático

Sem revisão no tempo certo, o conteúdo some da memória antes da prova.

Sem acompanhamento

Ninguém vê quem está ficando para trás até a nota chegar.

Persona

Jovem de 18 a 20 anos, recém-saído do ensino médio, que quer entrar em TI e sente dificuldade com os termos técnicos. Faz o nivelamento, revisa com flashcards e usa os simulados do ENADE; XP e moedas mantêm o ritmo.

São hipóteses: o próximo slide mostra de onde elas vieram, e o fim mostra como validá-las.

Como descobrimos o que o sistema precisava fazer

1. Análise de documentos

O Manual do PIM IV (o que cada etapa exige), os materiais das 31 disciplinas de AVA e ED do curso e as provas oficiais do ENADE de ADS (INEP).

2. Comparação com o Anki

O app de flashcards mais usado mostrou a revisão espaçada (SM-2) e os quatro botões de resposta.

3. Persona e cenários

A equipe é de alunos de ADS: a persona e os cenários de estudo partiram da nossa própria experiência de ingressantes.

4. Priorização

As necessidades viraram épicos e itens do Product Backlog (PB01 a PB22) no Trello, ordenados por sprint.

5. Protótipo e iteração

Cada entrega foi um sistema funcionando, revisado num pull request antes de entrar.

6. Validação

Testes de usabilidade com roteiro de tarefas, no computador e no celular.

Ainda falta: entrevistas e questionário com ingressantes reais, num piloto com uma turma.

Da necessidade até o teste

8
regras de negócio (RN01 a RN08)

14
requisitos funcionais (RF01 a RF14)

6
requisitos não funcionais (RNF01 a RNF06)

Exemplo de rastreio, de ponta a ponta:

Necessidade
"Quero estudar no ônibus, sem internet."

→

Requisito
RF12 e RNF05: o app estuda sem internet e envia depois.

→

Backlog
PB10 a PB12 (Mobile e sincronização), na Sprint 4.

→

Código
Fila de respostas no aparelho, enviada quando a conexão volta.

→

Teste
26 testes do app: envio com e sem internet e a fila.

Nex_TI: um site, um app e uma API, com o mesmo banco

43 fases de estudo, incluindo o ENADE	**1198** cartas, com resumos e fontes	**31** disciplinas de AVA e ED dos quatro semestres	**3** perfis: Aluno, Tutor e Administrador
27 rotas na API, em camadas	**16** casos de uso modelados no Astah	**452** testes automáticos aprovados	**8** disciplinas do semestre aplicadas

O conteúdo reúne 43 fases e 1198 cartas: além de ser feito com as disciplinas, o Nex_TI ensina um pouco de cada disciplina do curso.

Empreendedorismo em TI: o negócio por trás do sistema

O que a disciplina ensinou

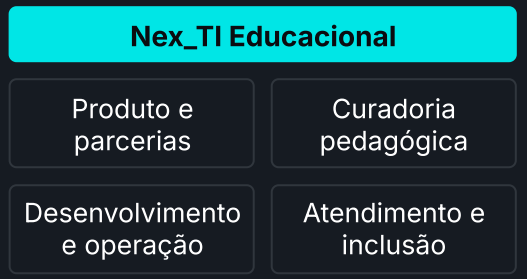
- Plano de negócios e proposta de valor.
- Ponto de equilíbrio e sustentabilidade.

ARAÚJO NETO (2026)

Onde usamos

- Nex_TI Educacional, uma EdTech fictícia: estudantes (B2C) e instituições (B2B).
- Quatro funções: produto e parcerias, curadoria pedagógica, desenvolvimento e operação, atendimento e inclusão.
- Hipóteses de receita e custo; o ponto de equilíbrio será calculado com dados do piloto.

Organograma proposto



Canvas do Nex_TI Educacional

Parcerias
Instituições de ensino com ações de nivelamento.

Atividades
Curadoria pedagógica; desenvolvimento e operação da plataforma; atendimento e inclusão.

Recursos
Site, app, API e banco; 1198 cartas com fontes; nuvem (Vercel e Azure); a equipe.

Proposta de valor
Estudo ativo dos fundamentos de TI, revisão no tempo certo e acompanhamento do tutor, numa experiência acessível.

Relacionamento
Tutor acompanhando o aluno; XP, moedas e níveis; o assistente Nexor.

Canais
Site no navegador e app no celular; as instituições parceiras.

Segmentos
Ingressantes de cursos de TI (B2C) e instituições de ensino (B2B).

Custos
Nuvem (API, banco e site), domínio, taxa de pagamento e divulgação; depois, curadoria e suporte. Hoje R$ 0 (planos gratuitos); ao escalar, cerca de R$ 202 por mês (estimativa).

Receitas (premissa)
Licença por aluno para instituições (R$ 5,00/mês) e assinatura individual (R$ 9,90/mês).

Os números do plano de negócios vêm nos próximos slides: são uma simulação, para o piloto confirmar.

SWOT: onde o Nex_TI está forte e onde precisa melhorar

Forças

- Site e app com a mesma API e o mesmo banco.
- Regras no servidor e 452 testes automáticos.
- Estudo sem internet no app.
- Conteúdo de 31 disciplinas, com fontes.

Fraquezas

- Ainda sem uso por alunos reais.
- A infraestrutura gratuita "dorme" sem uso.
- Viabilidade econômica não demonstrada.
- Curadoria administrativa e suporte formal a implementar.

Oportunidades

- Organizar percursos de estudo.
- Recursos acessíveis.
- Continuidade entre computador e celular.
- Ações de nivelamento das instituições.

Ameaças

- Baixa adesão dos alunos.
- Custos de operação.
- Inconsistência entre dispositivos.
- Necessidade de revisão pedagógica.

Plano de negócios: os 4 P's do Nex_TI

Produto

Plataforma Web e app de estudo de TI: flashcards com revisão espaçada, Trilha com 31 disciplinas, simulados do ENADE, gamificação e painel do Tutor.

Preço (premissa)

Instituições: R$ 5,00 por aluno ativo por mês. Alunos: R$ 9,90 por mês. O estudo básico continua gratuito, como na proposta social.

Praça

No navegador e no celular, inclusive sem internet; venda direta a instituições com ações de nivelamento.

Promoção

Parcerias com cursos de TI, demonstração para coordenações, redes sociais e indicação entre alunos.

Plano de negócios: o custo hoje e ao crescer

Investimento para escalar (estimativa)

ESPECIFICAÇÃO	R$
Domínio .com.br (primeiro ano)	40,00
Capital de giro (3 × R$ 202)	606,00
Divulgação inicial	300,00
Total	**946,00**

Custo fixo mensal

SERVIÇO	HOJE	AO ESCALAR (R$)
API	Plano gratuito	68,33
Banco de dados	Plano de aluno da faculdade (Azure)	26,00
Site	Vercel gratuito	104,00
Domínio	Ainda não comprado (endereço da Vercel)	3,33
Total por mês	**R$ 0,00**	**≈ 202,00**

Hoje o Nex_TI custa **R$ 0**: API e site em planos gratuitos, banco no plano de aluno e o endereço da Vercel. Os valores ao escalar são **estimativas** com preços públicos (US$ 1 = R$ 5,20).

Meta de vendas e os indicadores

Cenário estimado: o Nex_TI já nos planos pagos, cobrando de uma instituição e de assinantes.

META DO PILOTO (POR MÊS)	R$
1 instituição com 100 alunos × R$ 5,00 + 30 assinantes × R$ 9,90	797,00
(–) Taxa de pagamento (4%) e custo fixo	234,00
Lucro líquido (70,6%)	**563,00**

Break even

202,00 ÷ (5,00 × 0,96)
= 202,00 ÷ 4,80 = 42,1

43 alunos

Payback

946,00 ÷ 563,00
= 1,68

1,7 mês

Rentabilidade

563,00 ÷ 946,00 × 100
= 59,5%

≈ 60% ao mês

É um bom negócio? Pode ser, com escala

E se a equipe receber?

R$ 500 de pró-labore para cada um dos 5 sócios: custo fixo de 202 + 2.500 = **R$ 2.702 por mês**.
Novo ponto de equilíbrio: 2.702 ÷ 4,80 = **563 alunos**, cerca de 6 instituições com 100 alunos.

Conclusão

- O produto já existe e o investimento é pequeno (R$ 946).
- O piloto se paga com uma turma.
- Para virar empresa de verdade, precisa de várias instituições parceiras.

Próximo passo: um piloto com uma turma, para trocar as premissas pelos números reais. Premissas, contas e fontes em relatorio/docs/plano de negocios/PLANO_DE_NEGOCIOS.md.

Relações Étnico-Raciais e Afrodescendência: estudar sem barreiras

O que a disciplina ensinou

- Ações afirmativas.
- Lei 10.639/2003 e Estatuto da Igualdade Racial (Lei 12.288/2010).

TOLEDO-PLAÇA (2023)

Onde usamos

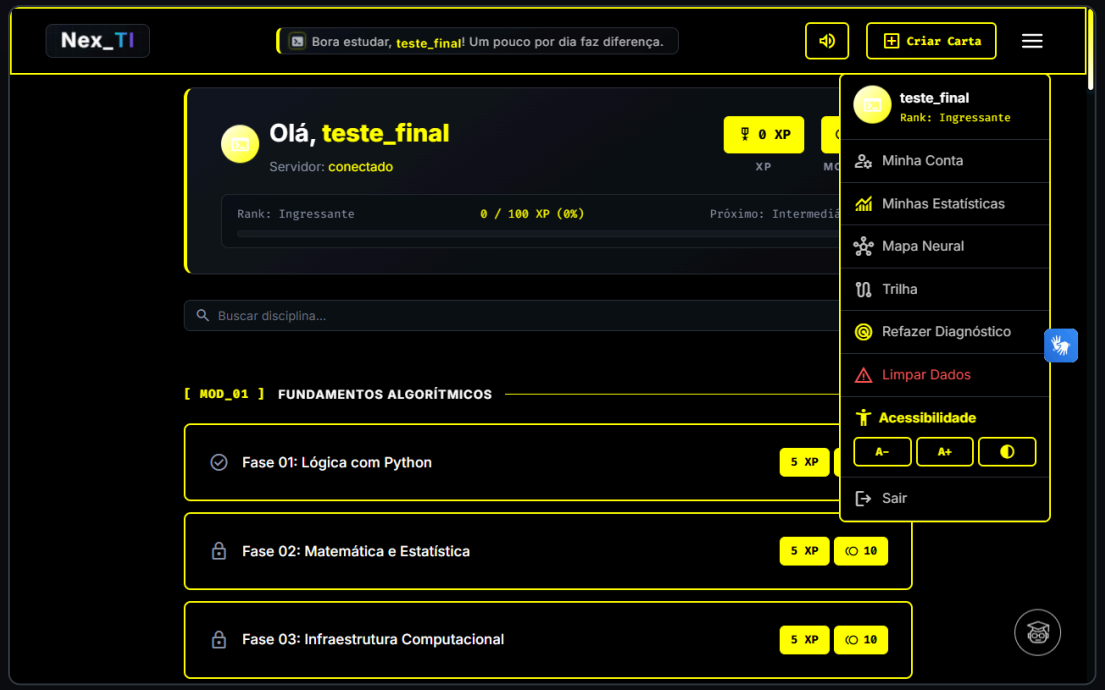
- Proposta: curadoria de conteúdo sem estereótipos, incluindo profissionais negros e afrodescendentes, com fontes.
- Proposta: política de uso e canal de relato contra discriminação.
- Estudo gratuito, inclusive sem internet no app.

Já no sistema

- VLibras, alto contraste e zoom da fonte.
- Tudo pelo teclado, com foco visível e janelas anunciadas ao leitor de tela.
- Botões de pelo menos 44 px no celular.

Acessibilidade ajuda, mas não substitui ações contra a discriminação: a política e o canal ainda precisam ser publicados e testados.

Acessibilidade de verdade, na tela



Alto contraste ligado pelo menu ☰ (tela real)

O que aparece

- **A- e A+:** aumentam ou diminuem o texto do site inteiro.
- **Contraste:** fundo preto e destaques em amarelo, para quem enxerga pouco.
- **VLibras** (a mão azul à direita): traduz o texto para Libras.
- Tudo também pelo teclado, com foco visível.

Desenvolvimento Web com .NET: o site e a API

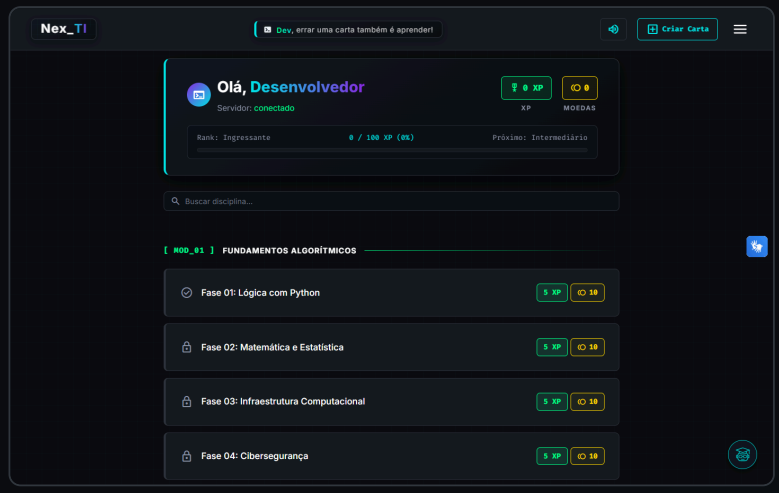
O que a disciplina ensinou

- Minimal APIs, EF Core e migrations.
- JWT, CORS, limite de requisições e OpenAPI.

PERES (2026)

Onde usamos

- API em .NET 10 com essas mesmas peças: 27 rotas REST.
- Login com JWT de 24 horas e limite de tentativas; o banco é criado pelas migrations.
- Site em React conversando com a API.



Painel do aluno (tela real)

Uma rota da API: Minimal API com JWT

C# sistema/apps/api/Endpoints/EndpointExtensions.cs

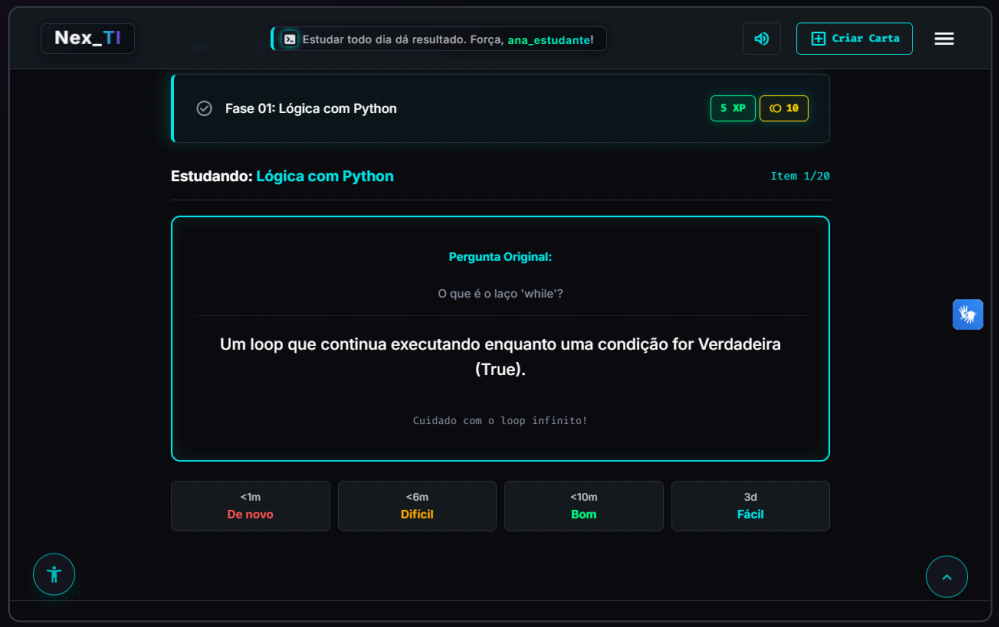
```
app.MapPost("/api/progresso/atualizar", async (ProgressoUpdate request,
IProgressoService progresso, ClaimsPrincipal userClaims) =>
{
    if (IdDoUsuario(userClaims) is not int currentUserId) return
Results.Unauthorized();
    if (request.UsuarioId > 0 && currentUserId != request.UsuarioId) return
Results.Forbid();

    return ParaHttp(await progresso.RegistrarRespostaAsync(currentUserId,
request.FlashcardId, request.Qualidade), Results.Ok);
}).RequireAuthorization();
```

O que a disciplina ensinou, aqui

- **Minimal API:** a rota é um `MapPost`.
- **JWT:** `RequireAuthorization()`; sem login, 401.
- Só o próprio aluno grava a resposta dele; outro usuário recebe 403.
- A rota não fala com o banco: chama a interface do serviço.

O estudo e a estrutura administrativa



Estudo: os quatro botões do SM-2



Acompanhamento dos alunos (Tutor e Admin)

SM-2

Errei volta amanhã; Difícil, Bom e Fácil afastam a revisão. XP e moedas: quem calcula é o servidor.

Tutor e Administrador

O Tutor acompanha os alunos e modera as cartas; o Administrador também define perfis.

Trilha e Nexor

31 disciplinas por módulo; o Nexor responde com os materiais da plataforma, sem serviço externo.

Desenvolvimento Mobile: o app que estuda sem internet

O que a disciplina ensinou

- Aplicações multiplataforma.
- React Native com Expo.
- Armazenamento local e acessibilidade.

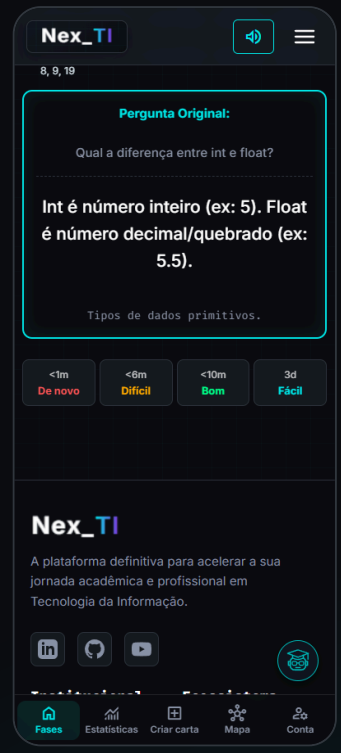
MONTEIRO (2026); BOZOLAN (2025)

Onde usamos

- App Expo com a mesma API e o mesmo banco do site.
- Token guardado no cofre do aparelho (SecureStore).
- Fila de respostas no AsyncStorage para estudar sem internet.
- 26 testes automáticos.



Painel



Carta e materiais

A fila do app: sem internet, nada se perde

JAVASCRIPT — sistema/apps/react-mobile/src/services/sincronizacao.js

```
async function enviarOuEnfileirar(item) {
  let resposta = null;
  try {
    resposta = await enviar(item);
  } catch (erro) {
    if (erro instanceof ErroDeRede) {
      if (item.tipo === 'stats') estado.fila = estado.fila.filter((i) => i.tipo
!== 'stats');
      estado.fila.push(item);
    } else if (erro instanceof ErroDaApi && erro.status === 401) {
      estado.fila.push(item);
      await salvar();
      throw erro;
    }
    // Outros erros da API (ex.: 400) não adiantam repetir: o item é descartado.
  }
```

Como funciona

- **Com internet:** a resposta vai direto à API.
- **Sem internet:** entra na fila, guardada no aparelho (AsyncStorage).
- **Sessão expirada (401):** guarda e volta ao login.
- Na próxima sincronização a fila é enviada, e o servidor recalcula o SM-2.

Programação Aplicada em .NET: orientação a objetos e camadas

O que a disciplina ensinou

Classes, herança, interfaces e encapsulamento; separar o programa em camadas com responsabilidades distintas.

Onde usamos

Rotas → serviços → banco: cada serviço tem uma interface, ligada por injeção de dependência; os perfis herdam (Usuário → Aluno → Tutor → Administrador).

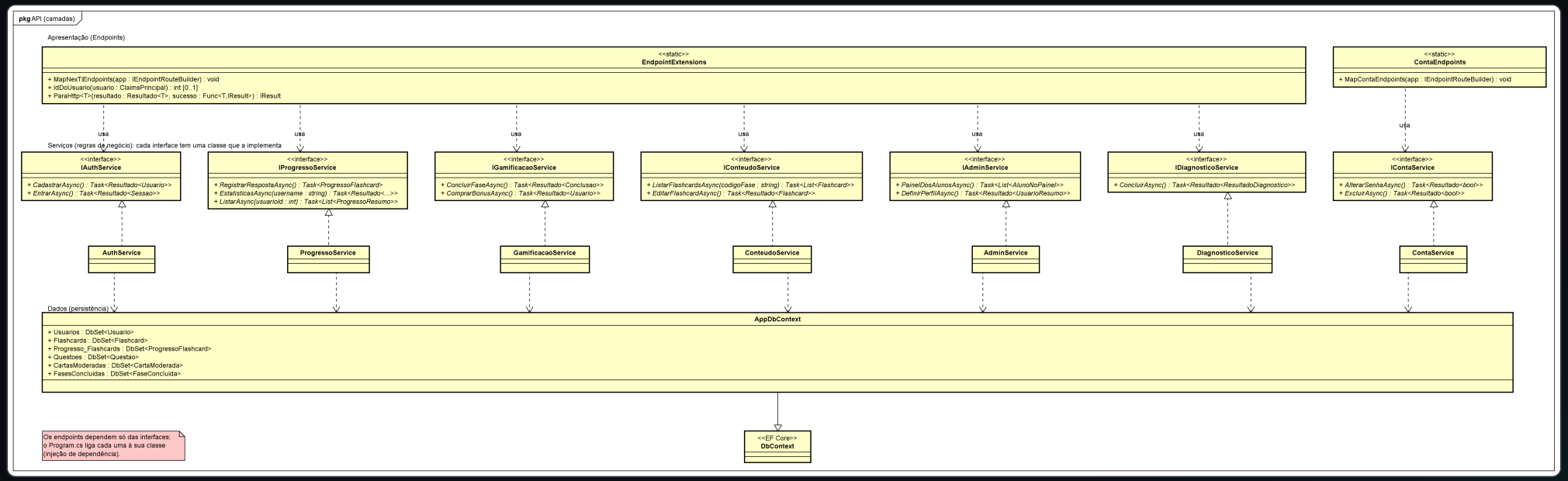


Diagrama de classes das camadas (Astah, gerado do código)

16 casos de uso, todos tirados do código

Para cada caso de uso, dois diagramas

- Um **diagrama de classes**: quem participa.
- Um **diagrama de sequência**: a ordem das mensagens.
- Um par para cada um dos 16 casos de uso; o exemplo vem no próximo slide.

Gerados do código, no Astah

Um gerador monta o arquivo do Astah a partir do código, com a explicação de cada elemento: mudou o código, gera de novo.

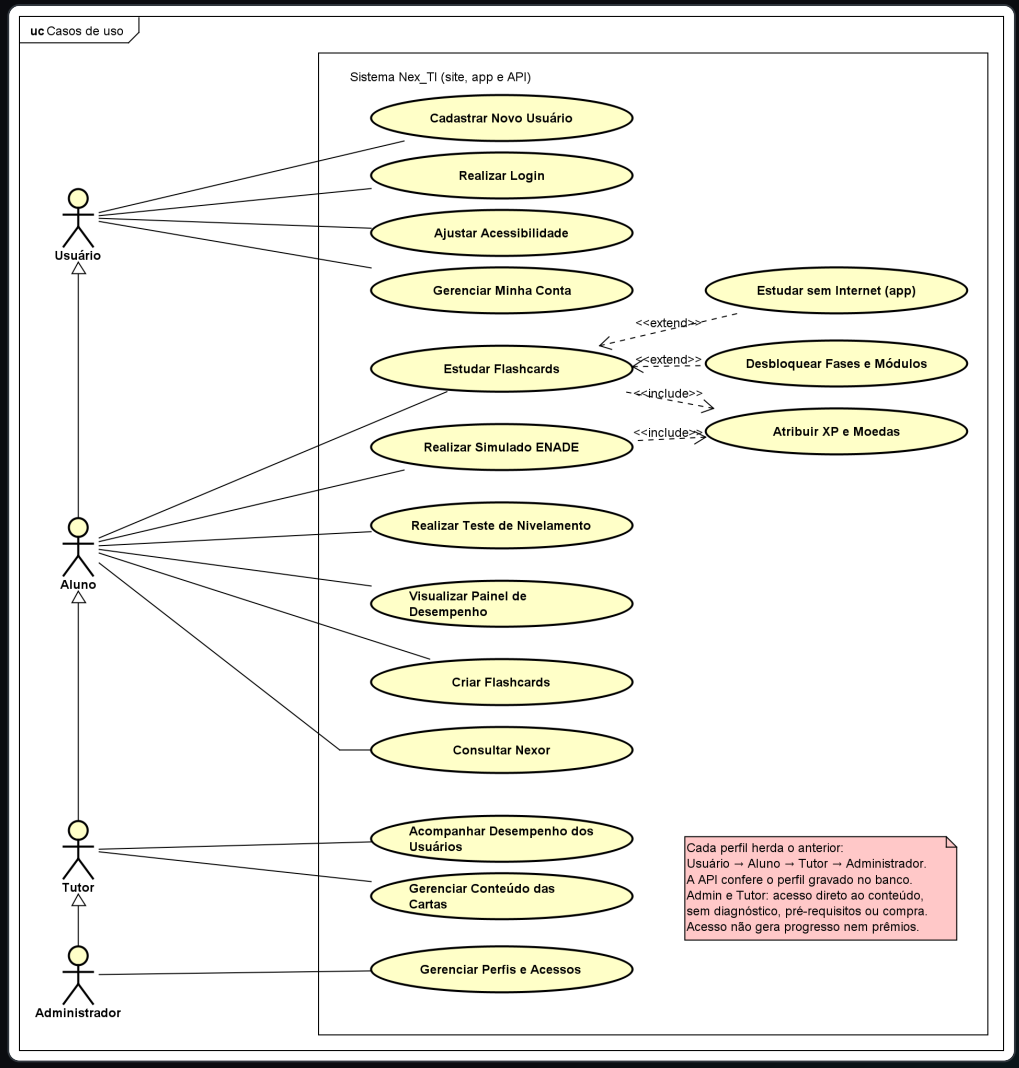
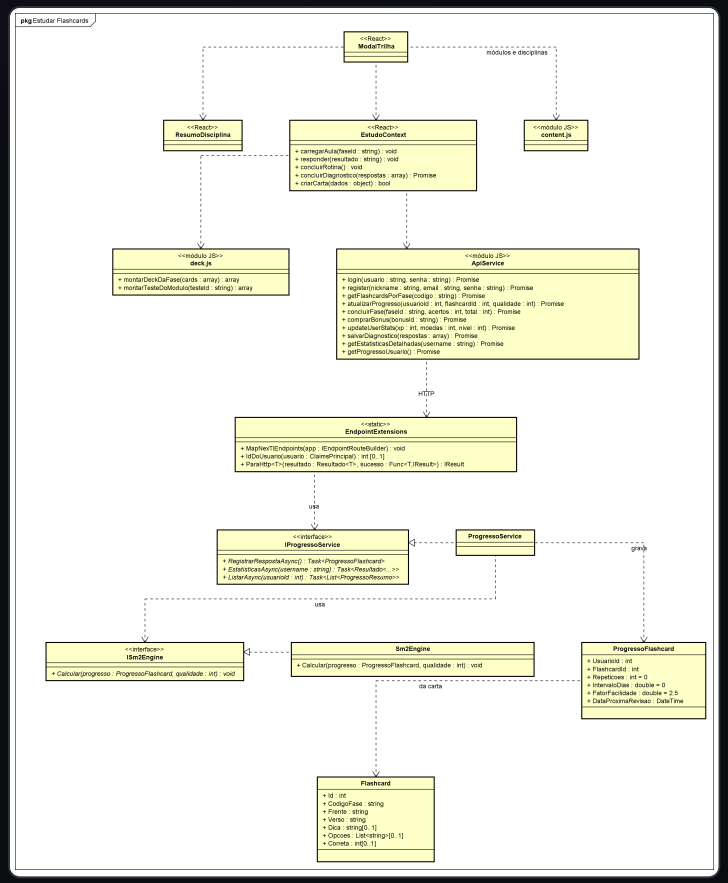
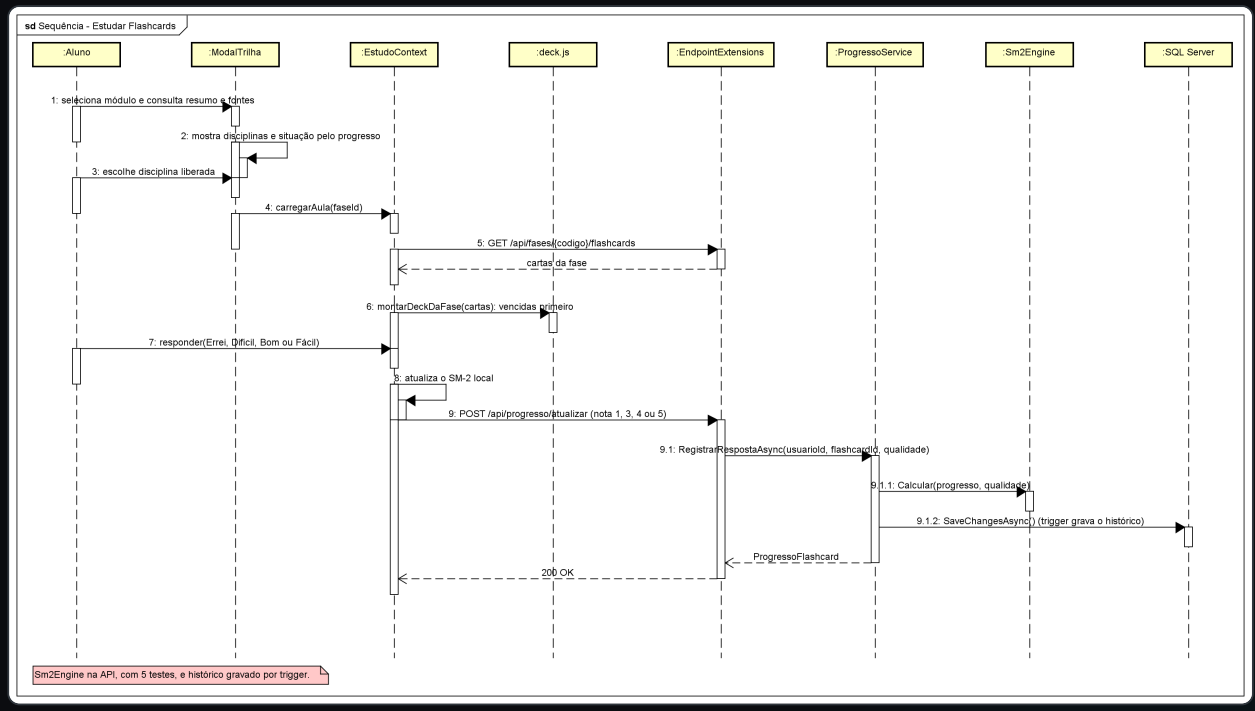


Diagrama de casos de uso

Exemplo: Estudar Flashcards, em classes e em sequência



Classes: quem participa (tela → API → serviço → entidade)



Sequência: a ordem das mensagens quando o aluno responde

Programação de Banco de Dados: regras dentro do banco

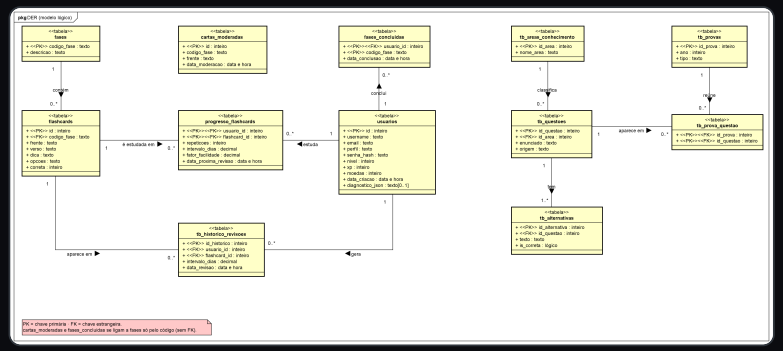
O que a disciplina ensinou

- Integridade referencial, joins e views.
- Procedures, triggers e funções.
- Backup e restore.

MARTINS (2026)

Onde usamos

- `tg_progresso_historico` grava cada revisão; `tg_usuarios_protege_gamificacao` impede que o XP diminua.
- `sp_ranking_alunos`, `sp_estatisticas_aluno`, `sp_cadastrar_flashcard` e views de ranking.
- Chaves estrangeiras e backup automático na Azure.



DER (modelo lógico)

Uma trigger: o XP nunca diminui

T-SQL — sistema/database/NexTI_DB.sql

```
CREATE OR ALTER TRIGGER tg_usuarios_protege_gamificacao
ON usuarios
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN deleted d ON d.id = i.id
        WHERE i.xp < d.xp OR i.nivel < d.nivel
    )
        THROW 50001, 'XP e nível do aluno não podem diminuir.', 1;
END;
```

O que a disciplina ensinou, aqui

- **Trigger AFTER UPDATE:** roda depois de cada alteração em `usuarios`.
- `inserted` tem o valor novo e `deleted`, o antigo.
- Se o XP ou o nível caírem, `THROW` desfaz a alteração.
- Vale até para quem mexer no banco por fora da API (regra RN05).

Cloud Computing e DevOps: no ar, com deploy automático

O que a disciplina ensinou

Modelos de serviço (IaaS, PaaS, SaaS), elasticidade, LGPD, contêineres, CI/CD e monitoramento.

REIS (2026)

Onde usamos

App Service e Azure SQL (PaaS), Docker, deploy automático a cada merge, Application Insights e os direitos do titular (exportar e excluir a conta).



Tudo em camadas gratuitas: depois de um tempo parado, a primeira requisição pode levar até um minuto.

O deploy automático: do merge ao Azure

GITHUB ACTIONS — .github/workflows/api-azure.yml (trecho)

```
on:
  push:
    branches: [main]

jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Testes
        run: dotnet test sistema/tests/NexTI_API.Tests/NexTI_API.Tests.csproj -c
Release

      - name: Publicar
        run: dotnet publish sistema/apps/api/NexTI_API.csproj -c Release -o ${{
runner.temp }}/api

      - name: Deploy no App Service
        uses: azure/webapps-deploy@v3

      - name: Conferir a API publicada
```

Como funciona (CI/CD)

- Cada merge na `main` dispara o GitHub Actions.
- **Testes primeiro:** se algum falhar, nada é publicado.
- Publica a API e faz o deploy no Azure App Service.
- No fim, chama a API publicada e só fica verde se ela responder.

Gerenciamento de Projetos Ágil: como a equipe trabalhou

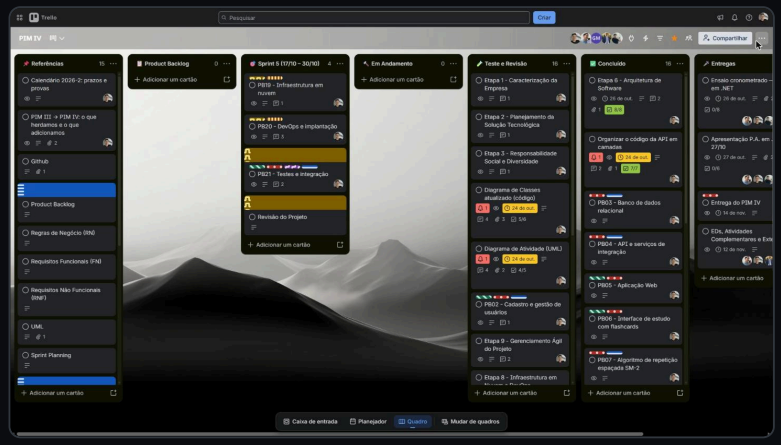
O que a disciplina ensinou

- Scrum e Kanban.
- Backlog e sprints.
- Caminho crítico (CPM) e organização do tempo.

ARANTES (2026a)

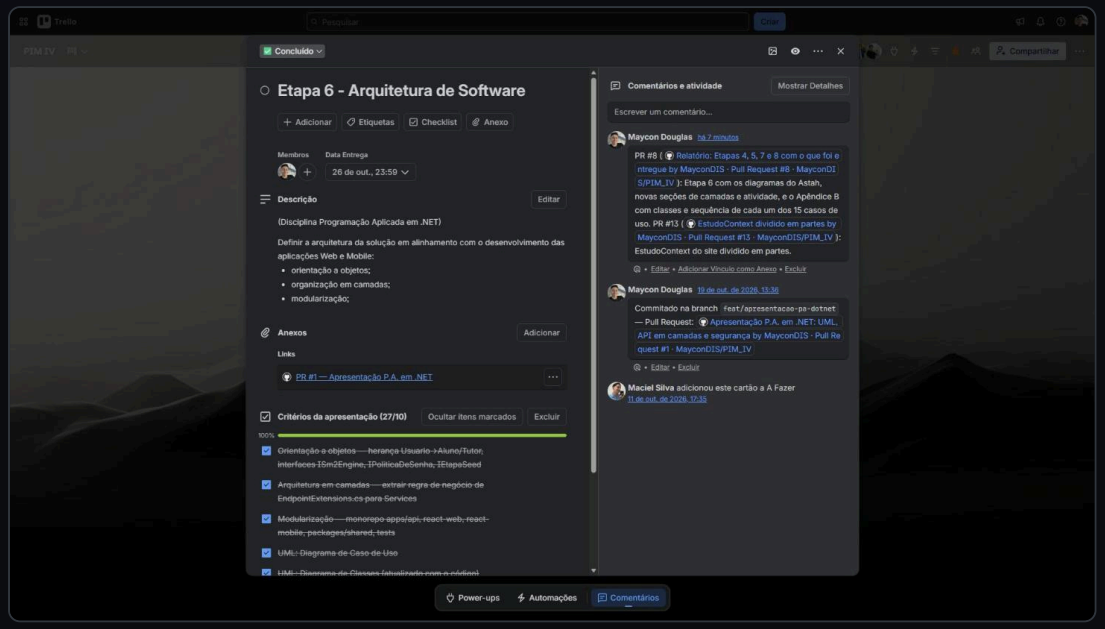
Onde usamos

- Product Backlog PB01 a PB22 no Trello.
- Cinco sprints, cada uma com meta e entrega.
- Cada cartão leva o link do pull request; caminho crítico até a entrega.



Quadro do PIM IV no Trello

Um cartão do Trello por dentro



Cartão da Etapa 6 no quadro do PIM IV

O que tem no cartão

- Responsável e prazo.
- Checklist com os critérios de pronto.
- O link do pull request que fez a entrega.
- As etiquetas de prioridade e de área.

Uma resposta do aluno passa pelas oito disciplinas

O aluno responde "Bom" numa carta, no celular, dentro do ônibus:

1. **Mobile**
   a resposta espera na fila do aparelho até a internet voltar.
2. **Web com .NET**
   chega à rota de progresso, com o token JWT conferido.
3. **Programação Aplicada em .NET**
   o serviço de progresso usa a interface do SM-2 para calcular a próxima revisão.
4. **Banco de Dados**
   o trigger grava a revisão no histórico.
5. **Cloud e DevOps**
   tudo roda no Azure, e o Application Insights registra a requisição.
6. **Projetos Ágil**
   essa função entrou pelo backlog (PB12), num pull request com teste.
7. **Empreendedorismo**
   a revisão no tempo certo é a proposta de valor da Nex_TI.
8. **Relações Étnico-Raciais**
   estudar sem internet amplia o acesso de quem tem conexão ruim.

452 testes aprovados, rodando a cada pull request

PARTE	TESTES
API (unitários, integração, caixa preta e caixa branca)	271
Pacote compartilhado (inclui carga)	124
App de celular	26
Programa desktop (P.A. em .NET)	31

Os testes acharam defeitos de verdade

- Carga: o intervalo do SM-2 sem teto passava do ano 9999.
- Caixa branca: gravação de resposta sem validar a nota.
- Usabilidade: estatísticas contavam fase liberada como concluída.

Todos corrigidos, com teste novo para não voltar.

Sincronia automática

Um script confere se os números do relatório, do README e destes slides batem com o código, e se os diagramas são os do Astah.

O que aprendemos trabalhando juntos

PROBLEMA	SOLUÇÃO
Trocar de tarefa custa tempo: com dois projetos ao mesmo tempo, cerca de 20% se perde na troca.	Um cartão por pessoa em "Em Andamento" e pull requests em sequência, um assunto por vez.
A integração do app não coube na Sprint 4.	A sprint tem prazo fixo: o item voltou ao backlog e foi priorizado de novo para a Sprint 5.
A Sprint 5 concentrou entregas.	Revisar a capacidade no planejamento de cada sprint e acompanhar o caminho crítico até a entrega.
A divisão planejada não se manteve: o desenvolvimento ficou concentrado.	Para o próximo projeto: tarefas pequenas com responsável e prazo no Trello, acompanhamento semanal e programação em dupla para passar o conhecimento.

Voltando ao problema: o que resolvemos e o que falta

Muitos termos novos

43 fases e 1198 cartas, com resumos e fontes das disciplinas do curso.

Estudo pouco sistemático

O SM-2 traz a carta de volta no tempo certo, no site e no app, até sem internet.

Sem acompanhamento

O Tutor vê o XP e as revisões de cada aluno.

Confirmado

A parte técnica funciona de ponta a ponta: mesma API e mesmo banco, regras no servidor, nuvem, testes e monitoramento.

Próximo passo

Completar a elicitação com ingressantes reais: entrevista, questionário e um piloto medindo o desempenho antes e depois.

Nex_TI

Demonstração e perguntas

Site

nexti-pim-iv.vercel.app

Código e documentação

github.com/MayconDIS/PIM_IV

Relatório

Relatório ABNT e versão em Word na pasta `relatorio/`

UNIP

UNIVERSIDADE PAULISTA

Obrigado!

Nex_TI · estudar TI no tempo certo

Gabriel Alves Moreira
H67HJI4

Maciel Costa da Silva
R280985

Maycon Douglas Inácio Silva
H719CD3

Miguel Angel Fernandez Ortiz
H7858F9

Rafael Mesquita
H6722I0